ΠΡΟΤΥΠΟ ΥΠΟΒΟΛΗΣ - 1η ΕΡΓΑΣΙΑ

CSE701 - Big Data / Ανάλυση Δεδομένων Μεγάλου Όγκου

Πεδίο	Συμπλήρωση
Επιλογή	Α - Πρακτική
Ονοματεπώνυμο/α	Πετρίδης Χρήστος
ΑΜ	ics24199
Ομάδα	Ατομική
Ημερομηνία	6/11/2026

Σύντομη περίληψη (150-250 λέξεις)

Σκοπός της παρούσας εργασίας είναι η ανάλυση δεδομένων καταγραφής περιήγησης (clickstream analytics) και η σχεδίαση αποδοτικών κατανεμημένων ροών επεξεργασίας βασισμένων στο προγραμματιστικό υπόδειγμα MapReduce. Με αφετηρία ένα συνθετικό σύνολο 20.000 γεγονότων από ηλεκτρονικό κατάστημα σε 8 ευρωπαϊκές χώρες, πραγματοποιήθηκε αρχικά λεπτομερής έλεγχος ποιότητας (data audit), όπου εντοπίστηκαν και αντιμετωπίστηκαν ελλείπουσες τιμές σε κρίσιμα πεδία, όπως το revenue και ο χρόνος απόκρισης. Στη συνέχεια, σχεδιάστηκαν και υλοποιήθηκαν σε περιβάλλον Python τρεις βασικοί αναλυτικοί αλγόριθμοι: ένας αλγόριθμος μονού σταδίου για την καταμέτρηση ενεργειών ανά χώρα και τύπο συσκευής, ένα pipeline δύο σταδίων για τον προσδιορισμό του κορυφαίου προϊόντος σε τζίρο ανά επικράτεια, καθώς και ένας αλγόριθμος ορθού υπολογισμού του μέσου χρόνου απόκρισης μέσω μεταφοράς ζευγών (sum, count) για την αποφυγή του μαθηματικού σφάλματος του μέσου των μέσων όρων. Όλα τα στάδια Map, Shuffle και Reduce επαληθεύτηκαν ανεξάρτητα με χρήση της βιβλιοθήκης pandas, επιτυγχάνοντας πλήρη ταύτιση αποτελεσμάτων. Τέλος, διερευνήθηκαν οι προκλήσεις μετάβασης σε πραγματική κλίμακα δισεκατομμυρίων γεγονότων, αναδεικνύοντας το κόστος μεταφοράς δεδομένων στο δίκτυο (Shuffle I/O) και την έντονη ανισοκατανομή φορτίου (data skew) ως τα κύρια σημεία συμφόρησης.

1. Πρόβλημα και απαιτήσεις

Τι ακριβώς ζητείται:

Αντικείμενο της εργασίας είναι η ανάλυση δεδομένων καταγραφής περιήγησης (clickstream logs) από ηλεκτρονικό εμπόριο και η ανάπτυξη κατανεμημένων αλγορίθμων βάσει του προγραμματιστικού υποδείγματος MapReduce. Ζητείται αρχικά ο διεξοδικός έλεγχος ποιότητας και εγκυρότητας του dataset (Data Audit) και η τεκμηριωμένη προεπεξεργασία του. Στη συνέχεια, ζητείται η σχεδίαση και υλοποίηση σε Python τριών διακριτών ροών MapReduce (Mapper → Shuffle/Grouping → Reducer):

1. **One-stage MapReduce:** Καταμέτρηση του πλήθους των events ανά συνδυασμό χώρας και τύπου συσκευής (country, device).
2. **Two-stage Pipeline:** Εύρεση για κάθε χώρα του προϊόντος (product_id) με το μέγιστο συνολικό τζίρο (revenue_eur) από πραγματικές αγορές (Job 1: συνάθροιση εσόδων ανά χώρα και προϊόν → Job 2: ανάδειξη μέγιστου ανά χώρα).
3. **Average Response Time:** Υπολογισμός του μέσου χρόνου απόκρισης (response_ms) ανά συσκευή μέσω κατανεμημένης μεταφοράς ζευγών (sum, count) για την αποφυγή μαθηματικού σφάλματος. Τέλος, απαιτείται η ανεξάρτητη επικύρωση όλων των αποτελεσμάτων με τη βιβλιοθήκη pandas, καθώς και η θεωρητική/αρχιτεκτονική ανάλυση κλιμάκωσης (scaling) για τη διαχείριση δισεκατομμυρίων events.

Ποια δεδομένα και απαιτήσεις είναι σημαντικά:

- **Δομή & Πεδία:** Το dataset περιλαμβάνει 20.000 εγγραφές και 17 πεδία. Κρίσιμα χαρακτηριστικά για τους υπολογισμούς είναι τα country (8 ευρωπαϊκές χώρες), device (mobile, desktop, tablet), event_type (view, search, add_to_cart, purchase), product_id, revenue_eur και response_ms.
- **Χαρακτηρισμός μέσω των 5Vs:**
    - Volume: Επεξεργασία 20.000 γραμμών ως δείγμα επικύρωσης λογικής (proof-of-concept), με πρόβλεψη κλιμάκωσης σε δισεκατομμύρια εγγραφές όπου απαιτείται οριζόντια επέκταση (scale-out) και διαμέριση (partitioning).
    - Velocity: Προσομοίωση συνεχούς ροής καταγραφών χρήστη (clickstream) που σε πραγματικές συνθήκες απαιτεί διαχωρισμό real-time απόκρισης από batch αναλύσεις.
    - Variety: Ημι-δομημένα logs που έχουν τυποποιηθεί σε tabular μορφή.
    - Veracity: Ύπαρξη ελλιπών τιμών (missing values) και έντονων ακραίων τιμών (outliers), που καθιστούν αναγκαία την εφαρμογή αυστηρών κανόνων καθαρισμού πριν την επεξεργασία.
    - Value: Εξαγωγή επιχειρησιακών μετρικών (εμπορική απόδοση προϊόντων ανά χώρα, κατανομή επισκεψιμότητας, εντοπισμός προβλημάτων latency).

Ποιες παραδοχές γίνονται (Assumptions):

- **Εγκυρότητα Συναλλαγών Revenue:** Θεωρείται ότι μόνο τα γεγονότα τύπου event_type == 'purchase' παράγουν έσοδα. Για τον υπολογισμό του κορυφαίου προϊόντος (A3), εξαιρούνται πλήρως οι 16 εγγραφές purchase με ελλιπές revenue_eur (λόγω NaN σε τιμή ή ποσότητα), θεωρώντας ότι οποιαδήποτε εκτίμηση/προσθήκη πλασματικής τιμής (imputation) θα νοθεύσει τα οικονομικά αποτελέσματα.
- **Φύση των Outliers στο Latency:** Οι μετρήσεις response_ms που φτάνουν έως και τα 24,8 δευτερόλεπτα θεωρούνται πραγματικά επιχειρησιακά περιστατικά φόρτου ή δικτύου (operational latency) και δεν διαγράφονται ως σφάλματα, καθώς αντικατοπτρίζουν την πραγματική εμπειρία χρήστη. Αντίθετα, οι 18 εγγραφές με κενό response_ms (NaN) αγνοούνται κατά τον υπολογισμό του μέσου όρου.
- **Μονάδα Ανάλυσης:** Κάθε γραμμή αντιπροσωπεύει ένα αυτόνομο ατομικό event και όχι μοναδικό χρήστη ή συνεδρία (event-level granularity).

2. Μεθοδολογία / Σχεδιασμός

Η μεθοδολογική προσέγγιση της εργασίας διαρθρώνεται σε διαδοχικά στάδια επεξεργασίας, διαχωρίζοντας τη φάση ελέγχου και καθαρισμού των δεδομένων από τις κατανεμημένες αναλυτικές ροές MapReduce, καταλήγοντας στην ανεξάρτητη επαλήθευση των αποτελεσμάτων.

Ανάλυση των Σταδίων Ροής

- **Στάδιο 1: Φόρτωση & Έλεγχος Δεδομένων (Data Audit):** Η ροή ξεκινά με την επιθεώρηση του raw συνόλου δεδομένων. Ελέγχεται η δομή του σχήματος (schema), οι τύποι δεδομένων (dtypes), η πληρότητα (completeness) μέσω καταγραφής των ελλειπουσών τιμών, καθώς και η μοναδικότητα (uniqueness) των πρωτευόντων κλειδιών (event_id).

- **Στάδιο 2: Εφαρμογή Κανόνων Καθαρισμού (Cleaning Rules):** Πριν από οποιονδήποτε μετασχηματισμό, ορίζονται ρητοί κανόνες:

    - Για τα εμπορικά ερωτήματα, απομονώνονται αποκλειστικά οι έγκυρες αγορές (purchase) που φέρουν θετικό και συμπληρωμένο revenue_eur.

    - Για τους υπολογισμούς απόδοσης συστήματος, διατηρούνται τα πραγματικά περιστατικά υψηλής καθυστέρησης (outliers), εξαιρώντας μόνο τις κενές καταγραφές NaN.

- **Στάδιο 3: Εκτέλεση MapReduce Pipelines:** Η αναλυτική επεξεργασία υλοποιείται αυστηρά στη βάση του τριπτύχου Mapper -> Shuffle/Grouping -> Reducer:

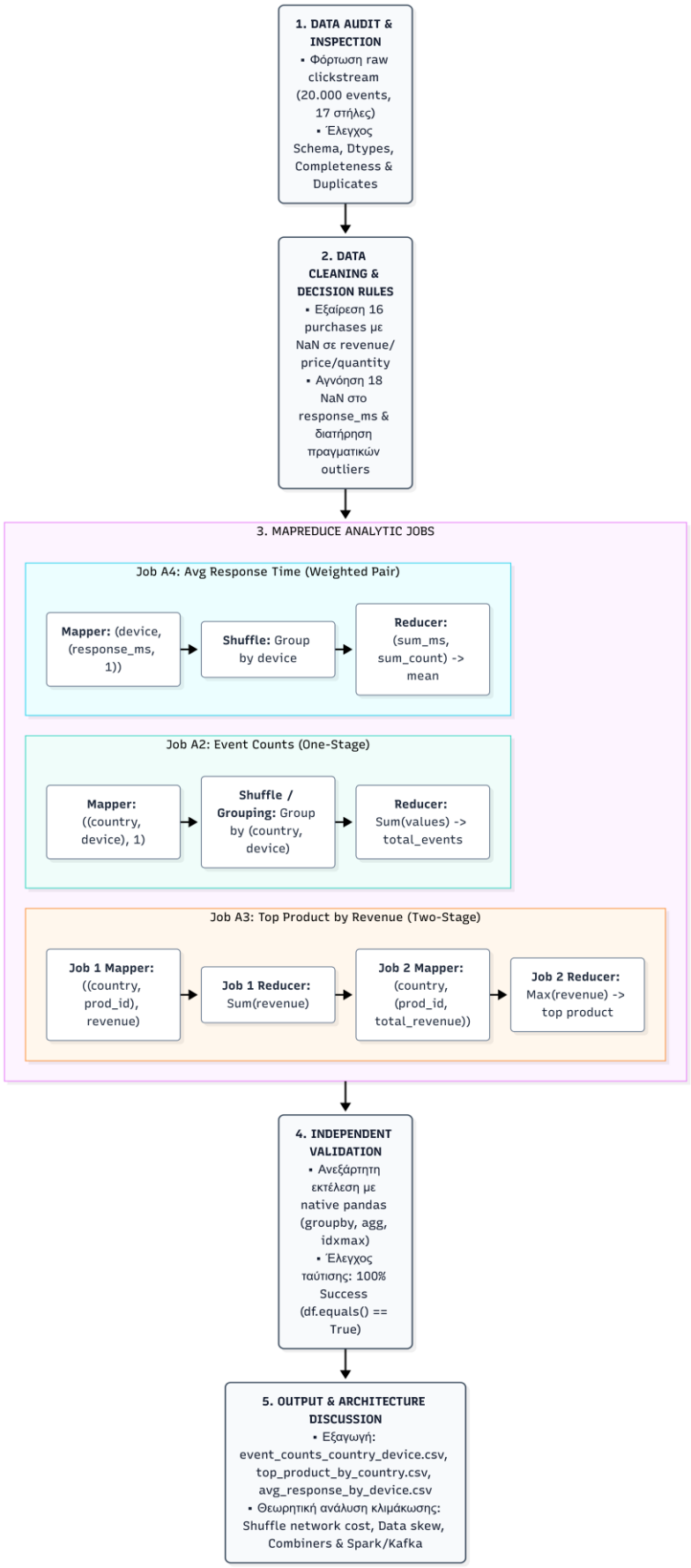
    - **Job A2 (Μονού Σταδίου):** Ορίζει ως σύνθετο κλειδί το ζεύγος (country, device) και εκπέμπει μονάδες (1). Το στάδιο Shuffle ομαδοποιεί τις καταγραφές και ο Reducer αθροίζει τα επιμέρους πλήθη.

    - **Job A3 (Δύο Σταδίων):** Απαιτεί ιεραρχική συγκέντρωση. Το **Job 1** αθροίζει το συνολικό τζίρο ανά συνδυασμό χώρας και προϊόντος ((country, product_id), revenue). Το **Job 2** αναδιατάσσει τα δεδομένα με κλειδί μόνο τη χώρα (country), μεταφέροντας ως τιμή την πληροφορία του προϊόντος και των συνολικών εσόδων του (product_id, total_revenue). Ο τελικός Reducer εφαρμόζει συνάρτηση μεγίστου (max) για να απομονώσει το κυρίαρχο προϊόν ανά επικράτεια.

    - **Job A4 (Σωστή Μέση Τιμή):** Για να αποφευχθεί το μαθηματικό σφάλμα της απλής μέσης τιμής επιμέρους μέσων όρων (average of averages), ο Mapper μεταφέρει ταυτόχρονα το άθροισμα και το πλήθος (response_ms, 1). Ο Reducer πραγματοποιεί συνάθροιση των δύο διαστάσεων ξεχωριστά ($\sum$ ms, $\sum$ count) και υπολογίζει το πηλίκο στο τελικό βήμα.

- **Στάδιο 4: Ανεξάρτητη Επικύρωση (Validation):** Κάθε MapReduce ροή ελέγχεται αντιπαραβάλλοντας τα τελικά εξαγόμενα DataFrames με ανεξάρτητες vectorized λειτουργίες της βιβλιοθήκης pandas (groupby(), idxmax(), mean()), διασφαλίζοντας 100% ταύτιση δεδομένων.

- **Στάδιο 5: Εξαγωγή Παραδοτέων & Μελέτη Κλιμάκωσης:** Παράγονται τα 3 τελικά αρχεία CSV και διατυπώνεται η αρχιτεκτονική ανάλυση για τη μετάβαση σε κατανεμημένο cluster πραγματικού μεγέθους.


1. DATA AUDIT & INSPECTION
▪ Φόρτωση raw clickstream (20.000 events, 17 στήλες)
▪ Έλεγχος Schema, Dtypes, Completeness & Duplicates
2. DATA CLEANING & DECISION RULES
▪ Εξαίρεση 16 purchases με NaN σε revenue/ price/quantity
▪ Αγνόηση 18 NaN στο response_ms & διατήρηση πραγματικών outliers
3. MAPREDUCE ANALYTIC JOBS
Job A4: Avg Response Time (Weighted Pair)
Mapper: (device, (response_ms, 1))
Shuffle: Group by device
Reducer: (sum_ms, sum_count) -> mean
Job A2: Event Counts (One-Stage)
Mapper: ((country, device), 1)
Shuffle / Grouping: Group by (country, device)
Reducer: Sum(values) -> total_events
Job A3: Top Product by Revenue (Two-Stage)
Job 1 Mapper: ((country, prod_id), revenue)
Job 1 Reducer: Sum(revenue)
Job 2 Mapper: (country, (prod_id, total_revenue))
Job 2 Reducer: Max(revenue) -> top product
4. INDEPENDENT VALIDATION
▪ Ανεξάρτητη εκτέλεση με native pandas (groupby, agg, idxmax)
▪ Έλεγχος ταύτισης: 100% Success (df.equals() == True)
5. OUTPUT & ARCHITECTURE DISCUSSION
▪ Εξαγωγή: event_counts_country_device.csv, top_product_by_country.csv, avg_response_by_device.csv
▪ Θεωρητική ανάλυση κλιμάκωσης: Shuffle network cost, Data skew, Combiners & Spark/Kafka

3. MapReduce σχεδιασμός

Στοιχείο	Περιγραφή
Input record (εγγραφή εισόδου)	**A2:** Κάθε γραμμή clickstream event (country, device, κ.λπ.) από το σύνολο των 20.000 εγγραφών. **A3 (Job 1):** Έγκυρες εγγραφές αγορών (event_type == 'purchase', με μη κενά product_id και θετικό revenue_eur). **A3 (Job 2):** Ενδιάμεσα αποτελέσματα Job 1: ((country, product_id), total_revenue). **A4:** Εγγραφές με έγκυρη μέτρηση καθυστέρησης (response_ms.notna()).
Key (κλειδί)	**A2:** Σύνθετο κλειδί (country, device) (π.χ. ('GR', 'mobile')). **A3 (Job 1):** Σύνθετο κλειδί (country, product_id) (π.χ. ('GR', 'P01043')). **A3 (Job 2):** Απλό κλειδί χώρας country (π.χ. 'GR'). **A4:** Κλειδί τύπου συσκευής device (π.χ. 'desktop').
Value (τιμή)	**A2:** Ο ακέραιος 1 (ένδειξη εμφάνισης ενός γεγονότος). **A3 (Job 1):** Το ποσό εσόδων revenue_eur της μεμονωμένης αγοράς. **A3 (Job 2):** Σύνθετη τιμή προϊόντος και αθροιστικού τζίρου (product_id, total_revenue). **A4:** Σύνθετο αριθμητικό ζεύγος (response_ms, 1) (χρόνος απόκρισης και πλήθος μονάδας).
Mapper (αντιστοίχιση)	**A2:** mapper_event_count(row) -> ((row['country'], row['device']), 1). **A3 (Job 1):** mapper_j1(row) -> ((row['country'], row['product_id']), row['revenue_eur']). **A3 (Job 2):** mapper_j2(item) -> (item[0][0], (item[0][1], item[1])), δηλαδή country -> (product_id, total_rev). **A4:** mapper_avg_response(row) -> (row['device'], (row['response_ms'], 1)).
Shuffle / Grouping (αναδιάταξη / ομαδοποίηση)	**A2:** (country, device) -> [1, 1, 1, ...] (24 διακριτά κλειδιά). **A3 (Job 1):** (country, product_id) -> [rev_1, rev_2, ...] (1.506 διακριτά κλειδιά). **A3 (Job 2):** country -> [(prod_1, rev_1), (prod_2, rev_2), ...] (8 κλειδιά χωρών).

Στοιχείο	Περιγραφή
	A4: device -> [(ms_1, 1), (ms_2, 1), ...] (3 κλειδιά συσκευών).
Reducer (συγκέντρωση)	**A2:** Άθροιση των μονάδων: sum(values) → συνολικά events ανά συνδυασμό. **A3 (Job 1):** Άθροιση εσόδων: sum(values) → συνολικό revenue ανά χώρα και προϊόν. **A3 (Job 2):** Αναζήτηση μέγιστης τιμής: max(tuples, key=lambda x: x[1]) → ανάδειξη του κορυφαίου προϊόντος ανά χώρα. **A4:** Ξεχωριστή συνάθροιση: sum(pair[0]) και sum(pair[1]), ακολουθούμενη από διαίρεση.
Output (έξοδος)	**A2:** 24 γραμμές με στήλες [country, device, event_count] (event_counts_country_device.csv). **A3:** 8 γραμμές με στήλες [country, product_id, total_revenue] (top_product_by_country.csv). **A4:** 3 γραμμές με στήλες [device, avg_response_ms] (avg_response_by_device.csv).
Validation (επικύρωση)	**A2:** df.groupby(['country', 'device']).size() → SUCCESS (100% ταύτιση). **A3:** groupby(['country', 'product_id'])['revenue_eur'].sum() και idxmax() → SUCCESS (100% ταύτιση). **A4:** valid_df.groupby('device')['response_ms'].mean() → SUCCESS (100% ταύτιση).

4. Αποτελέσματα / Worked examples

4.1 Αποτελέσματα Job A2: Κατανομή Events ανά (Country, Device)

Η ροή μονού σταδίου επεξεργάστηκε και τις 20.000 εγγραφές εισόδου, δημιουργώντας 24 μοναδικά κλειδιά (8 χώρες × 3 συσκευές). Τα αποτελέσματα εξήχθησαν στο αρχείο event_counts_country_device.csv.

Έλεγχος Επικύρωσης (Validation Check): Ο ανεξάρτητος υπολογισμός μέσω df.groupby(['country', 'device']).size() παρήγαγε πανομοιότυπο πίνακα 24×3, με τη σύγκριση event_counts_df.equals(pandas_a2) να επιστρέφει True (100% ταύτιση).

Χώρα	Desktop	Mobile	Tablet	Σύνολο ανά Χώρα	Ποσοστό (%)
BG	416	901	83	1.400	7,00%
CY	508	950	103	1.561	7,80%
DE	917	1.715	153	2.785	13,92%
ES	654	1.259	132	2.045	10,22%

FR	683	1.350	121	2.154	10,77%
GR	2.057	3.955	415	6.427	32,14%
IT	719	1.408	121	2.248	11,24%
RO	426	854	100	1.380	6,90%
Σύνολο	6.380	12.392	1.228	20.000	100,00%

4.2 Αποτελέσματα Job A3: Κορυφαίο Προϊόν ανά Χώρα βάσει Revenue

Από τις 1.604 συνολικές αγορές, απομονώθηκαν οι 1.581 έγκυρες συναλλαγές με θετικό και μη κενό τζίρο. Το Job 1 παρήγαγε 1.506 μοναδικά ενδιάμεσα ζεύγη (country, product_id). Στη συνέχεια, το Job 2 ομαδοποίησε ανά χώρα και εντόπισε το προϊόν με το μέγιστο συνολικό έσοδο. Τα αποτελέσματα αποθηκεύτηκαν στο top_product_by_country.csv.

Έλεγχος Επικύρωσης (Validation Check): Η εκτέλεση της pandas αλυσίδας δύο βημάτων valid_purchases.groupby(['country', 'product_id'])['revenue_eur'].sum() ακολουθούμενη από idxmax() ανά χώρα επικύρωσε απόλυτα τις παραπάνω τιμές (top_product_df.equals(p_j2) == True).

Χώρα (Country)	Κορυφαίο Προϊόν (Product ID)	Συνολικά Έσοδα (€)
BG	P02460	€1.000,56
CY	P02196	€650,24
DE	P00718	€658,09
ES	P01819	€376,02
FR	P00612	€295,50
GR	P01043	€1.175,10
IT	P00027	€387,06
RO	P02307	€618,46

4.3 Αποτελέσματα Job A4: Μέσος Χρόνος Απόκρισης ανά Συσκευή

Ο υπολογισμός βασίστηκε στις 19.982 εγγραφές με συμπληρωμένο χρόνο απόκρισης (αγνοήθηκαν 18 κενές τιμές). Τα τελικά αποτελέσματα εξήχθησαν στο avg_response_by_device.csv.

Έλεγχος Επικύρωσης (Validation Check): Η εντολή valid_df.groupby('device')['response_ms'].mean().round(2) ταυτίστηκε πλήρως με την έξοδο του Reducer (avg_response_df.equals(pandas_a4) == True).

Τύπος Συσκευής (Device)	Πλήθος Events (N)	Συνολικός Χρόνος (∑ms)	Μέσος Χρόνος Απόκρισης (ms)
Desktop	6.376	1.863.176,00	292,22 ms
Mobile	12.380	3.450.771,00	278,74 ms
Tablet	1.226	330.612,00	269,67 ms

4.4 Worked Example: Μαθηματική Ορθότητα Κατανεμημένου Μέσου Όρου

Για την αιτιολόγηση της επιλογής εκπομπής ζευγών (sum, count) έναντι του εσφαλμένου «μέσου των μέσων όρων» (average of averages), εξετάζεται το ακόλουθο υποθετικό σενάριο δύο υπολογιστικών κόμβων (Worker Nodes) που επεξεργάζονται αιτήματα συσκευών mobile:

- **Worker 1 (Μικρό partition):** Επεξεργάζεται $n_1 = 1$ αίτημα με τιμή 100 ms.
  - Τοπικός μέσος: $\bar{x}_1 = 100$ ms.
- **Worker 2 (Μεγάλο partition):** Επεξεργάζεται $n_2 = 9$ αιτήματα με άθροισμα 90 ms (μέσος όρος 10 ms ανά request).
  - Τοπικός μέσος: $\bar{x}_2 = 10$ ms.

Σύγκριση Αποτελεσμάτων:

1. **Λανθασμένη προσέγγιση (Average of Averages):**

   Αν κάθε κόμβος υπολόγιζε αυτόνομα το μέσο και ο Reducer έβγαζε το μέσο όρο των αποτελεσμάτων τους:

$$\bar{x}_{\text{wrong}} = \frac{\bar{x}_1 + \bar{x}_2}{2} = \frac{100 + 10}{2} = 55 \text{ ms}$$

2. **Ορθή κατανεμημένη προσέγγιση (Weighted Sum / Count):**

   Ο Mapper και ο Combiner διατηρούν τα επιμέρους βάρη εκπέμποντας $(100,1)$ και $(90,9)$, επιτρέποντας στον τελικό Reducer να υπολογίσει:

$$\bar{x}_{\text{correct}} = \frac{\sum \text{ms}}{\sum \text{count}} = \frac{100 + 90}{1 + 9} = \frac{190}{10} = 19 \text{ ms}$$

Η απλή μέση τιμή υπερεκτιμά το αποτέλεσμα κατά σχεδόν 190% (55 ms αντί για 19 ms), αποδεικνύοντας ότι ο μέσος όρος δεν είναι προσεταιριστική συνάρτηση και καθιστώντας υποχρεωτική τη μεταφορά των αθροιστικών στατιστικών $(sum, count)$ στο MapReduce.

5. Scaling, κόστος και περιορισμοί

Η μετάβαση από το δοκιμαστικό δείγμα των 20.000 εγγραφών σε πραγματική κλίμακα παραγωγής (π.χ. 20 δισεκατομμύρια γεγονότα / ημέρα, μεγέθους πολλαπλών TBs) καθιστά την εκτέλεση σε μία μηχανή (single-machine scale-up) αδύνατη λόγω εξάντλησης RAM, CPU cores και Disk I/O. Η υιοθέτηση κατανεμημένης αρχιτεκτονικής (scale-out) επιτρέπει οριζόντια κλιμάκωση, εισάγει όμως εγγενείς περιορισμούς, κόστη δικτύου και σημεία συμφόρησης.

1. Κόστος Shuffle & Μεταφορά Δεδομένων (Network I/O)

- **Ανάλυση:** Το Shuffle αποτελεί την πιο δαπανηρή φάση στο υπόδειγμα MapReduce. Κατά τη φάση αυτή, τα ενδιάμεσα ζεύγη κλειδιού-τιμής που παράγουν οι Mappers πρέπει να σειριοποιηθούν, να γραφούν προσωρινά στον τοπικό δίσκο και να μεταφερθούν μέσω του δικτύου προς τους αντίστοιχους Reducers βάσει του κλειδιού.

- **Περιορισμοί στα Jobs:**

  - Στο **Job A2 (Event Counts)**, χωρίς τοπική προ-ομαδοποίηση, 20 δισεκατομμύρια ζεύγη ((country, device), 1) θα διέσχιζαν το δίκτυο του cluster, προκαλώντας network saturation.

  - Στο **Job A3 (Two-Stage Pipeline)**, το Job 1 απαιτεί shuffle για όλα τα purchase events ανά (country, product_id). Το Job 2, ωστόσο, διακινεί εξαιρετικά μικρότερο όγκο (μόνο τα μοναδικά

αθροιστικά ζεύγη ανά χώρα), επιβεβαιώνοντας ότι το pipeline μειώνει προοδευτικά το ενδιάμεσο αποτύπωμα.

- Στο **Job A4 (Latency)**, διακινούνται ζεύγη (device, (response_ms, 1)) για κάθε έγκυρη μέτρηση.

2. Ο Ρόλος του Combiner (Map-Side Pre-Aggregation)

- **Μηχανισμός:** Ο Combiner λειτουργεί ως τοπικός Reducer εντός του ίδιου κόμβου (Mapper node) πριν τη φάση μετάδοσης στο δίκτυο.
- **Εφαρμογή στα Jobs:**
  - **Job A2:** Αντί ο Mapper να εκπέμπει εκατομμύρια tuples ((country, device), 1), ο Combiner αθροίζει τοπικά τα πλήθη, περιορίζοντας τα εκπεμπόμενα ζεύγη σε το πολύ 24 (8 χώρες $\times$ 3 συσκευές) ανά worker.
  - **Job A3 (Job 1):** Μειώνει δραστικά το μέγεθος διακίνησης αθροίζοντας τοπικά τα έσοδα ανά (country, product_id).
  - **Job A4:** Καθώς ο μέσος όρος δεν είναι προσεταιριστική πράξη, ο Combiner **δεν** εκτελεί διαίρεση, αλλά αθροίζει μερικά αθροίσματα και πλήθη: (device, (local_sum_ms, local_count)). Έτσι, στέλνονται μόλις 3 ζεύγη ανά worker κόμβο, εκμηδενίζοντας το I/O.

3. Data Skew & Stragglers (Ανισοκατανομή)

- **Πρόβλημα:** Σε ένα κατανεμημένο σύστημα, ένα job ολοκληρώνεται μόνο όταν τερματίσει και η πιο αργή εργασία (straggler task). Όταν ορισμένα κλειδιά συγκεντρώνουν δυσανάλογα μεγάλο όγκο δεδομένων, ο κόμβος που τα διαχειρίζεται υπερφορτώνεται (skew).
- **Ευρήματα στο Dataset:**
  - *Γεωγραφικό Skew:* Η Ελλάδα (GR) αντιπροσωπεύει το **32,14%** όλων των γεγονότων, ενώ χώρες όπως η Ρουμανία (RO) μόλις το **6,90%**. Ένας Reducer που λαμβάνει τα δεδομένα της Ελλάδας θα εκτελέσει πενταπλάσιο φόρτο.
  - *Συσκευές:* Το mobile συγκεντρώνει σχεδόν το **62%** της συνολικής κίνησης.
  - *Προϊόντα:* Ακολουθούν κατανομή νόμου δύναμης (Power-law), όπου λίγα δημοφιλή προϊόντα συγκεντρώνουν τη συντριπτική πλειονότητα των πωλήσεων.

4. Partitioning & Data Locality

- **Storage Partitioning:** Στο επίπεδο αποθήκευσης (Data Lake / S3 / HDFS), τα δεδομένα πρέπει να οργανώνονται ιεραρχικά:

/raw/clickstream/year=2026/month=09/day=01/country=GR/

Αυτό εξασφαλίζει **Partition Pruning**, επιτρέποντας στα αναλυτικά queries να σκανάρουν μόνο τους σχετικούς υποφακέλους και όχι ολόκληρο το ιστορικό TBs.

- **Compute Partitioning (Salting):** Για την αντιμετώπιση του Data Skew κατά το Shuffle, εφαρμόζεται τεχνική «αλατίσματος» (Salting): τα βαριά κλειδιά σπάνε σε υπο-κλειδιά (π.χ. GR_1, GR_2, GR_3) με τυχαία κατανομή σε πολλαπλούς Reducers, και ακολουθεί ένα δεύτερο ελαφρύ στάδιο συγκέντρωσης.

- **Data Locality:** Μεταφορά του υπολογισμού κοντά στα δεδομένα («move computation to data») ώστε ο Worker να εκτελείται στον ίδιο φυσικό server ή rack όπου είναι αποθηκευμένα τα αντίστοιχα blocks, εξαλείφοντας δια-δικτυακές καθυστερήσεις ανάγνωσης.

5. Fault Tolerance & Αξιοπιστία Κλίμακας

- **Αρχή Σχεδιασμού:** Σε ένα cluster εκατοντάδων κόμβων, οι αποτυχίες υλικού (δίσκοι, RAM, switch, servers) είναι καθημερινό φαινόμενο («failures are normal»).

- **Αντιμετώπιση:**

    - *Storage Layer:* Αυτόματο replication των blocks δεδομένων (συνήθως replication factor 3 σε διαφορετικά racks) ή χρήση Object Storage με υψηλή ανθεκτικότητα (durability 99.999999999%).

    - *Compute Layer:* Deterministic task re-execution. Εάν ένας worker καταρρεύσει κατά τη διάρκεια ενός Map task, το σύστημα αναθέτει αυτόματα το ίδιο task σε άλλον κόμβο που έχει τοπικό αντίγραφο του block, χωρίς επανεκκίνηση ολόκληρου του pipeline.

6. Trade-offs Αρχιτεκτονικής & Προτεινόμενες Βελτιώσεις

- **Scale-up vs Scale-out:** Το scale-up προσφέρει απλότητα χωρίς network overhead, αλλά προσκρούει γρήγορα σε οικονομικά όρια (hardware ceiling). Το scale-out παρέχει θεωρητικά απεριόριστη κλιμάκωση, αλλά επιβαρύνεται με distributed complexity (συντονισμός, διαχείριση αστοχιών, network bandwidth).

- **MapReduce (Disk-based) vs Apache Spark (In-Memory):** Το κλασικό MapReduce γράφει υποχρεωτικά όλα τα ενδιάμεσα αποτελέσματα στο δίσκο μεταξύ Jobs, προκαλώντας υψηλό Disk & Network I/O. Αντίθετα, σύγχρονες μηχανές όπως το **Apache Spark** κρατούν τα ενδιάμεσα datasets στη μνήμη (RAM) μέσω RDDs/DataFrames, επιτυγχάνοντας πολλαπλάσια ταχύτητα στα πολυσταδιακά pipelines.

- **CSV vs Columnar Parquet:** Η διατήρηση raw αρχείων CSV συνεπάγεται υψηλό αποθηκευτικό κόστος και ανάγνωση ολόκληρης της γραμμής. Η μετατροπή σε **Apache Parquet με Snappy συμπίεση** προσφέρει column projection (διάβασμα μόνο των απαραίτητων στηλών, π.χ. response_ms, παραλείποντας τις υπόλοιπες 16) και συμπίεση άνω του 70%, ελαχιστοποιώντας ταυτόχρονα το κόστος αποθήκευσης και μεταφοράς.

6. Συμπεράσματα

- **Τι μάθαμε:**
    - Η πρακτική εμπειρία ανέδειξε ότι η έννοια των Big Data δεν καθορίζεται απλώς από τον όγκο των gigabytes, αλλά από τις απαιτήσεις κλίμακας και τους περιορισμούς που επιβάλλουν τα 5Vs.
    - Επιβεβαιώθηκε στην πράξη ο κανόνας «Inspect before clean»: τα δεδομένα δεν καθαρίζονται αυθαίρετα (π.χ. με τυφλό fillna(0)), καθώς κάθε μετασχηματισμός ενσωματώνει επιχειρησιακές παραδοχές.
    - Κατανοήθηκε σε βάθος το προγραμματιστικό μοντέλο MapReduce (Mapper → Shuffle/Grouping → Reducer) ως ένας κατανεμημένος μηχανισμός split -> apply -> combine, ο οποίος επιτρέπει την υλοποίηση σύνθετων αναλύσεων (καταμετρήσεις, ιεραρχικά pipelines δύο σταδίων και σταθμισμένους μέσους όρους) με πλήρη μαθηματική συνέπεια.
- **Βασικό τεχνικό συμπέρασμα:**
    - Η κατανεμημένη επεξεργασία δεν συνεπάγεται «δωρεάν ταχύτητα» («Distributed ≠ δωρεάν ταχύτητα: κερδίζω κλίμακα, πληρώνω πολυπλοκότητα»).
    - Σε μεγάλη κλίμακα, το βασικό σημείο συμφόρησης (bottleneck) δεν είναι η επεξεργαστική ισχύς της CPU, αλλά το **κόστος του Shuffle (Network & Disk I/O)** σε συνδυασμό με το **Data Skew** (ανισοκατανομή κλειδιών, όπως το 32% της Ελλάδας και το 62% του mobile).

    - Αποδείχθηκε ότι η προσθήκη περισσότερων υπολογιστικών κόμβων δεν επιλύει έναν κακό σχεδιασμό («περισσότεροι κόμβοι δεν διορθώνουν ένα κακό data layout ή αλγόριθμο»). Η χρήση αποδοτικών μηχανισμών όπως οι τοπικοί Combiners (map-side aggregation), η διατήρηση διανυσματικών ζευγών (sum, count) για μη προσεταιριστικές πράξεις, καθώς και η στρατηγική διαμέρισης (Partitioning / Salting), αποτελούν προαπαιτούμενα για την αποφυγή καθυστερήσεων από straggler workers.
- **Πότε θα αλλάζαμε προσέγγιση:**
    - Από Single-Machine (Pandas) σε Distributed: Εάν το μέγεθος των δεδομένων ξεπεράσει τη διαθέσιμη μνήμη RAM (working set limit) ή αν απαιτείται διατήρηση ιστορικού πολλών ετών (π.χ. 20 δισεκατομμύρια events), η χρήση Python/Pandas σε μία μηχανή εγκαταλείπεται υποχρεωτικά υπέρ κατανεμημένων συστημάτων.
    - Από MapReduce σε Apache Spark: Για επαναληπτικές αναλύσεις ή πολυσταδιακά pipelines (όπως το Job A3), η κλασική υλοποίηση MapReduce αποδεικνύεται ασύμφορη λόγω της συνεχούς εγγραφής ενδιάμεσων αρχείων στο δίσκο. Σε αυτή την περίπτωση, η μετάβαση σε **Apache Spark** είναι επιβεβλημένη, καθώς διατηρεί τα δεδομένα στη μνήμη (in-memory processing) και βελτιστοποιεί αυτόματα το Shuffle I/O.
    - Από Batch σε Streaming (Apache Kafka / Spark Streaming): Εάν οι επιχειρησιακές απαιτήσεις μεταβληθούν από ημερήσια αναλυτική επεξεργασία (nightly batch ETL) σε παρακολούθηση λειτουργίας σχεδόν σε πραγματικό χρόνο (near-real-time dashboard με latency $\leq 10 - 60$ sec), η batch προσέγγιση αντικαθίσταται από streaming αρχιτεκτονική βασισμένη σε event logs (Kafka) για συνεχή απορρόφηση και ροϊκή επεξεργασία συμβάντων.

Βιβλιογραφία / Πηγές

Γ. Αγγελίδης, *CSE701: Big Data / Ανάλυση Δεδομένων Μεγάλου Όγκου – Εκπαιδευτικό Υλικό & Διαφάνειες Μαθημάτων 1, 2 και 3*, Τμήμα Εφαρμοσμένης Πληροφορικής, Πανεπιστήμιο Μακεδονίας, Open eClass, 2026. Διαθέσιμο: https://openeclass.uom.gr/modules/document/index.php?course=DAI186&openDir=%2F6ad86030VVwu

J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, Jan. 2008. Διαθέσιμο: https://research.google/pubs/pub62/

M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster Computing with Working Sets," in *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 10)*, Boston, MA, 2010. Διαθέσιμο: https://www.usenix.org/legacy/event/hotcloud10/tech/full_papers/Zaharia.pdf

Apache Software Foundation, "Hadoop MapReduce Next Generation – MapReduce Architecture and Tutorial," *Apache Hadoop Documentation*, 2024. Διαθέσιμο: https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html

J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," in *Proc. 6th International Workshop on Networking Meets Databases (NetDB)*, Athens, Greece, 2011. Διαθέσιμο: https://www.microsoft.com/en-us/research/wp-content/uploads/2017/09/Kafka.pdf

The Pandas Development Team, "pandas: powerful Python data analysis toolkit documentation (GroupBy & Split-Apply-Combine)," *pandas Documentation*, 2024. Διαθέσιμο: https://pandas.pydata.org/docs/user_guide/groupby.html

Δήλωση χρήσης εργαλείων ΤΝ

☐ Δεν χρησιμοποιήθηκαν εργαλεία ΤΝ.

☑ Χρησιμοποιήθηκαν εργαλεία ΤΝ για**:** διερεύνηση ιδεών αρχιτεκτονικής κλιμάκωσης και μαθηματική επικύρωση σύνθετων σταδίων MapReduce

Σύντομη περιγραφή χρήσης:

Η χρήση εργαλείων ΤΝ (Gemini) έγινε σε ελάχιστα σημεία υψηλής δυσκολίας και αποκλειστικά σε ρόλο συμβούλου/thought partner. Συγκεκριμένα, αξιοποιήθηκε για τη θεωρητική διερεύνηση των trade-offs κλιμάκωσης σε δισεκατομμύρια γεγονότα (τεχνικές salting για την αντιμετώπιση του έντονου data skew, αρχιτεκτονικές storage vs compute) και για την ανεξάρτητη επιβεβαίωση της μαθηματικής ορθότητας της μετάδοσης ζευγών (sum, count) έναντι του μέσου των μέσων όρων.